

in28minutes.com · Interview Preparation Series

DOCKER

Interview Guide

10 Chapters · 78 Questions & Answers · June 2026

Table of Contents

1	Docker Fundamentals	<i>6 Q&A</i>
2	Building Docker Images	<i>13 Q&A</i>
3	Managing Docker Containers	<i>5 Q&A</i>
4	Persisting Data in Docker	<i>8 Q&A</i>
5	Docker Networking	<i>7 Q&A</i>
6	Docker Security	<i>9 Q&A</i>
7	Docker Compose	<i>6 Q&A</i>
8	Advanced Docker Concepts	<i>5 Q&A</i>
9	Docker Cheat Sheet	<i>9 Q&A</i>
10	Docker Interview - Diagrams	<i>10 Q&A</i>

CHAPTER 1

Docker Fundamentals

6 Questions & Answers

Q1**What are the challenges with Traditional Deployment Process?**

- Multiple setup steps: Setting up OS, software, and dependencies repeatedly can be tedious
- Time-consuming process: Manually performing these steps across multiple environments (Dev, QA, Stage, Prod) takes a lot of effort
- Lack of automation: Traditional methods lack automation
- High chance of mistakes: Numerous manual steps and separate documents increase the likelihood of errors
- Different documents: Each application - depending on its type (Java, Python, JavaScript) - needs different kind of setup instructions

Q2**Can you explain the sequence of steps in creating and deploying a Container Image?**

- Build Container Image: Create a Container image of your application with necessary components
- Push Image to Container Registry: Upload your created image to the registry for later use.
- Deploy it to Your Environment: Pull the image from the registry and run your application
Same approach in Local, DEV, QA, Stage, Production,...

Q3**What are the Advantages of using Docker?**

- Standardized Deployment Package: Same packaging approach irrespective of programming language
- Standardized Deployment Process: Run any Container image the same way, regardless of its contents
- Standardized Operations: Collect logs, metrics, and traces the same way, regardless of the container
- Lightweight Alternative to VMs: Uses fewer resources, making it faster and more efficient

Q4**Compare Virtual Machines (VM) and Containers**

Feature	VMs
Architecture	Complete virtualized computers with their own OS, running on a hypervisor
Performance	Resource-intensive - Each VM contains full OS
Boot Time	Slower, full OS startup
Resource Usage	Consumes more CPU, memory, storage
Image Size	Larger, includes a full OS
Security	Stronger, separate OS for each VM

Q5

Explain Important Components in Container Architecture

- **Container Image:** A package representing a specific version of your application, containing everything needed to run it

Complete Package: Includes the Operating System, Application Runtime, and Application Code & Dependencies

Consistent Deployment: Ensures your app runs the same way everywhere, from your local machine to cloud servers

- **Container Engine:** The core software that runs and manages containers

Run your Container Images: Creates containers

- **Container Registry:** Storage location for Container images

Examples: Docker Hub, Amazon Elastic Container Registry, Google Artifact Registry, Azure

Container Registry

Collaboration: Facilitates easy distribution of container images across teams

Integration: Works with CI/CD systems for automated building and deployment

Discovery: Acts as a discovery mechanism for official and community images

- **Container Repository:** A collection of Container images for a specific app

Q6

Why is Containerization important for DevOps and Microservices?

What is Containerization?

- **Definition:** Package app + dependencies into one container
- **Goal:** Run app the same everywhere – dev, test, prod
- **Tools:** Popular tool is Docker. Alternatives include containerd, runc, ..

Why is it Important in DevOps?

- Consistent Environments: Same setup from dev to prod
- "Works on My Machine" Solved: Containers avoid environment mismatch

Containerization Simplifies Deployment

- Faster Setup: No need to install libraries manually
- Less Errors: Fewer issues due to missing dependencies
- Quick Start: Run app with one command

Containerization Promotes Automation

- Automated Image Creation: Automatically build container images
- Works with CI/CD: Integrate with tools like Jenkins or GitHub Actions
- Supports Orchestration: Use Kubernetes to manage many containers

Containerization Simplifies Microservices Architectures

- Create microservice specific container images: Create different container images for each of the microservices
 - Same Process For Different Technology: Same process irrespective of technology used in a microservice - Java, Python, Node.js or ..
 - Scale Easily: Add or remove containers as needed
 - Containers Improve Observability: Get Logs, Metrics and Traces for different types of applications the same way

Containers are Platform Neutral

- Same Image Everywhere: Build once, run anywhere
- Run Anywhere: Works on local, server, or cloud
- Cloud Ready: Compatible with AWS, Azure, GCP

Containers are Resource Efficient

- Lightweight: Host OS is shared
- Faster Start-up: Containers launch quickly
- Higher Density: Run more containers (than VMs) on same hardware

CHAPTER 2

Building Docker Images

13 Questions & Answers

Q1

What is a Dockerfile? How do you build a Docker image?

Dockerfile: A file containing a series of instructions on how to build a Docker image

Example:

- Dockerfile
- Build the image using:
- Create container from the image:

Dockerfile

```
# Use an official Python runtime as the base image
FROM python:3.8-slim

# Sets the working directory inside the container
# Conceptually similar to `cd` command
WORKDIR /app

# Copies files/directories from the host to the container
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Exposes a port for external access
# Make port 80 available to the world outside this container
EXPOSE 80

# Define the command to run the application
CMD ["python", "app.py"]
```

Dockerfile

```
docker build -t my-python-app .
```

Dockerfile

```
docker run -d -p 80:80 my-python-app

# Test
curl http://localhost
```


Q2

What is a Base Image?

- Starting Point for Builds: Provides an initial starting point to build your container images
- Add Custom Things: Add your application code, dependencies, and configurations
- Huge Variety: You have a variety of Base Images

OS Images: Examples include ubuntu:22.04, debian:12, alpine:3.20;

Language Runtimes: Examples include openjdk:28-jdk-slim, node:30-alpine, python:4.12-slim; these come with pre-installed runtime environments

Minimal & Secure Options: Such as alpine:3.20 or gcr.io/distroless/java21; they are tiny and have a smaller attack surface

Blank Slate: The 'scratch' image gives a completely empty starting point for maximum control

Q3

Why is the Base Image a Key Choice?

- Size: Smaller images lead to faster downloads and less storage usage
- Security: Choosing a minimal and well-maintained image reduces vulnerability risks
- Performance: Influences startup time and runtime efficiency of your container
- Community Support: Popular images often have better documentation and troubleshooting resources

Q4

What is the difference between ENTRYPOINT and CMD in a Dockerfile? explain with example

- ENTRYPOINT: Defines the main executable
 - Main executable: Sets the container's primary command
 - Not easily overridden: Not overridden by arguments unless --entrypoint is used.
- CMD: Default arguments to the ENTRYPOINT
 - Default execution: Provides default arguments for container run
 - Can be easily overridden: Can be overridden by arguments passed to docker run.

ENTRYPOINT Example:

- ENTRYPOINT instruction sets the entry point for the container as python app.py
- When the container starts, it will run python app.py by default, and you cannot change this behavior by passing arguments to docker run
- To override the ENTRYPOINT, you need to use the --entrypoint flag with docker run:

ENTRYPOINT + CMD:

- This Dockerfile sets ENTRYPOINT to python and CMD to app.py

When the container starts, it will run python app.py

You can override the CMD argument like this:

CMD Alone Example:

- The CMD instruction specifies that the container should run python app.py by default
- You can override this command by specifying a different command when you run the container

Dockerfile

```
#!/ Use an official Python runtime as a parent image
FROM python:3.8-slim

# Set the working directory
WORKDIR /app

# Copy the current directory contents into the container
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Set the entrypoint to run the Python interpreter with app.py
ENTRYPOINT ["python", "app.py"]
```

Dockerfile

```
docker run --entrypoint /bin/bash myimage
```

Dockerfile

```
# Use an official Python runtime as a parent image
FROM python:3.8-slim

# Set the working directory
WORKDIR /app

# Copy the current directory contents into the container
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Set the entrypoint to run the Python interpreter
ENTRYPOINT ["python"]

# Define the default argument to run
CMD ["app.py"]
```

Dockerfile

```
docker run myimage another_script.py
```

Dockerfile

```
docker run myimage another_script.py
```

Dockerfile

```
# Use an official Python runtime as a parent image
FROM python:3.8-slim

# Set the working directory
WORKDIR /app

# Copy the current directory contents into the container
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Define the default command to run when the container starts
CMD ["python", "app.py"]
```

Dockerfile

```
docker run myimage python another_script.py
```

Q5

Difference between ADD and COPY in a Dockerfile

- COPY: Simply copies files and directories from the host to the container
- ADD: More powerful. Can copy files and directories and also supports URL downloads and automatic extraction of compressed files

Example:

Dockerfile

```
# Copy a local file into the /app directory in the image
COPY localfile.txt /app/

# Download a file from a URL
# and unzip it into the /app directory in the image
ADD http://example.com/file.tar.gz /app/
```

Q6

How do you tag a Container image? Why is tagging important?

- Tagging Command: Use `docker build -t imagename:tag .` to assign a tag

- Importance:

Version Control: Helps identify image versions

`microservice-a:v1,microservice-a:v2,microservice-a:v3`

`mysql:5.7, mysql:6.0`

`openjdk:8.0.0,openjdk:25.0.0`

Rollbacks: Easily revert to older versions of your image if newer ones fail

Flexibility: Gives teams flexibility w.r.t. deploying different versions in different environments (dev, staging, prod).

Multiple Tags: You can have multiple tags for an image

`latest` to indicate the latest version. Whoever want to use the latest version always can use the `latest` tag. `myapp:latest`

Example:

Dockerfile

```
# Build an myapp image with a tag 1.0.0
docker build -t myapp:1.0.0 .

# You can have multiple tags for an image
# Add additional tag latest to myapp image with tag 1.0.0
docker tag myapp:1.0.0 myapp:latest
```

Q7

Explain the layered approach to building a Container image?

- Layered builds: Container images are created in layers
- Example: Most instructions in a Dockerfile, such as `FROM`, `RUN`, `COPY`, create a new, read-only layer on top of the previous one.
- Why is this important?
 - Layers are cached: Layers are cached for faster and more efficient builds
 - However Be Cautious: Changing one layer triggers a rebuild of subsequent layers
 - Example Below: In Changed Dockerfile, Every layer from Instruction 2 CHANGED is rebuilt again

Example Dockerfile

Changed Dockerfile

Code

```
Instruction 1
Instruction 2
Instruction 3
Instruction 4
Instruction 5
Instruction 6
```

Code

```
Instruction 1
Instruction 2 CHANGED
Instruction 3
Instruction 4
Instruction 5
Instruction 6
```

Q8

How can you design Dockerfiles to maximize layer reuse?

- Application code changes often: In applications, code changes often, libraries/dependencies change infrequently
 - Application code evolves to add new features, fix bugs, and adapt to changing requirements
 - Libraries and dependencies, on the other hand, tend to have more stable APIs and are updated less frequently, often for security patches, performance improvements, or new features
- Recommended: Build Layers with Libraries and Dependencies First and Code Later

Non-Optimized Dockerfile:

- In this Dockerfile, the COPY . . command copies the entire application code (app.js,...) + dependencies files (package.json and package-lock.json) before running npm install
 - This means any change in the application code (even a small one) will cause Docker to invalidate the cache for the COPY . . instruction and all subsequent instructions, leading to frequent rebuilds of the npm install step

Optimized Dockerfile

- In this optimized Dockerfile, package.json and package-lock.json files are copied first
- The RUN npm install step runs only if dependencies change (package.json or package-lock.json)
 RUN npm install will NOT if ONLY application code (app.js,...) changes!
- The COPY . . command copies the rest of the application code after installing dependencies.
- This ensures changes to the application code do not affect the dependencies layer, allowing Docker to cache the npm install step unless package.json or package-lock.json change

Dockerfile

```
# Use an official Node.js runtime as a parent image
FROM node:24

# Set the working directory
WORKDIR /usr/src/app

# Copy application code + dependencies files
COPY . .

# Install dependencies - TAKES TIME!
RUN npm install

# Expose the application port
EXPOSE 3000

# Run the application
CMD ["node", "app.js"]
```

Dockerfile

```
# Use an official Node.js runtime as a parent image
FROM node:24

# Set the working directory
WORKDIR /usr/src/app

# Copy only package.json and package-lock.json first
COPY package*.json ./

# Install dependencies - TAKES TIME!
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the application port
EXPOSE 3000

CMD ["node", "app.js"]
```

What is a multi-stage build, and how can it be used to reduce the size of a container image?

- Scenario: Imagine needing a full compiler or SDK to build your app — but once it's built, you only need the tiny binary to run it. Do you really want to ship all that extra baggage in production? There's a better way.
- Multi-Stage Build: A Docker technique that lets you use multiple images in one Dockerfile — one for building your app and another for running it
- Build Big, Run Small: Use a larger image (like node, maven, or golang) to compile your app — then copy just the final output into a much smaller runtime image
- Reduces Image Size Dramatically: Final image contains only what's necessary to run the app — no compilers, dev tools, or source code
- Improves Security by Design: Smaller images have fewer vulnerabilities because they include fewer packages and libraries
- Makes CI/CD More Efficient: Smaller images mean faster downloads, deployments, and less storage usage across environments
- How Does It Work?:
 - Multiple FROM statements: Each FROM starts a new build stage
 - Artifact copying: Transfer only necessary artifacts between stages to keep the final image small

Golang Example 1: Larger Image Size Final image includes unnecessary build tools, making it larger than needed

Golang Example 1: Multi Stage Build Separate build stage, copy only compiled binary to minimal base

Dockerfile

```
FROM golang:1.16
WORKDIR /app
COPY main.go .
RUN go build main.go
CMD ["/main"]
```

Dockerfile

```
# Stage 1: Build the Go application
FROM golang:1.16 AS builder

WORKDIR /app

COPY main.go .

RUN go build main.go

# Stage 2: Create a minimal image with the built binary
FROM alpine:latest # MINIMAL BASE IMAGE

WORKDIR /root/

COPY --from=builder /app/main . # COPY WHAT WE WANT!

CMD ["/main"]
```

Q10

Is it possible to make changes to an existing Container image?

- Read-only design: Container images are immutable — you cannot directly modify them. This ensures consistency, reproducibility, and follows the principle of immutable infrastructure.
- Commit changes: However, Modifications can be made to a running container and then committed to create a new image
 - New image creation: Instead of updating the existing image, a new one is created, following immutable infrastructure principles

Example: Commit changes from container & Create new image

- Step 1: Run a container from the nginx:latest image
- Step 2: Modify the running container
- Step 3: Create new container image from running container
- Step 4: Run a container from the newnginx:modified image (newly created)

Dockerfile

```
docker run --name test1 -d -p 80:80 nginx

# Test
curl http://localhost:80
```

Dockerfile


```
docker exec test1 sh -c \  
'echo "<h1 style="font-size:400px">Testing</h1>" > /usr/share/nginx/html/index.html'  
  
# Test  
curl http://localhost:80
```

Dockerfile

```
docker commit test1 newnginx:modified
```

Dockerfile

```
docker run --name test2 -d -p 81:80 newnginx:modified  
  
# Test  
curl http://localhost:81
```

Q11

What is the purpose of the .dockerignore file?

- Exclude files: .dockerignore prevents specific files and directories from being copied into the Container image during the build and make image lightweight
- Pattern matching: Define file and directory patterns (similar to .gitignore) that Docker should skip during the build process

Example .dockerignore File:

Code

```
# Ignore node_modules directory  
node_modules  
  
# Ignore log files  
*.log  
  
# Ignore development or editor-specific files  
.DS_Store  
.vscode/  
.idea/
```

Q12

What Are Alternatives to Writing Dockerfile?

- **Manual Dockerfile Challenges:** Writing good Dockerfile is challenging
Mistakes can lead to security issues, large images and long build times (inefficient caching)
- **Jib:** Java-specific tool that eliminates the need for Dockerfiles
Layer Caching: Built-in optimization to speed up image building process
Security Features: Reduce risks by automating best practices
- **Buildpacks:** Multi-language support (Java, Node.js, Python, etc.), no Dockerfile required
Layer Caching: Built-in optimization to speed up image building process
Security Features: Reduce risks by automating best practices
Language Compatibility: Supports a variety of programming languages

Code

```
# Build a Java image with JIB
mvn compile com.google.cloud.tools:jib-maven-plugin:build \
-Dimage=gcr.io/my-project/my-app

# Build a Java image with packeto build pack
pack build my-spring-app \
--builder paketobuildpacks/builder:base \
--env BP_JVM_VERSION=24 \
--env BP_LOG_LEVEL=debug

# Use Spring Boot Maven Plugin (Uses packeto build pack)
mvn spring-boot:build-image
```

Q13

What is OCI and Why Is It Important?

- **Problem Before OCI:** Proprietary formats led to vendor lock-in and limited interoperability
- **Open Container Initiative(OCI):** Defines open, vendor-neutral specifications for containers
Image Spec: Standard format for container images
Runtime Spec: Standardization for running containers across different platforms
Distribution Spec: Standardized methods for pushing and pulling images through container registries (Supported by Docker Hub, Google Artifact Registry, Amazon Elastic Container Registry, ...)
- **Open Governance:** Managed by the Linux Foundation, ensuring open collaboration and transparency
- **OCI is Supported By Almost All Container Based Tools and Cloud Platforms:** Docker, Podman, Buildpacks, Kubernetes, containerd, AWS, Azure, Google Cloud

CHAPTER 3

Managing Docker Containers

5 Questions & Answers

Q1

Quick Review of Container Image Creation and Usage

- 1: Build Image: `docker build -t my-app:0.1 .`
- 2: Push to Container Registry: `docker push my-app:0.1`
- 3: Pull from Container Registry: `docker pull my-app:0.1` [OPTIONAL]
- 4: Run a Container: `docker run my-app:0.1`
- NOTE: If image does not exist on the host, daemon would pull the image first and then run it
- NOTE: A set of images are cached on the host. Daemon would check the cache before pull the image again.

Q2

Explain important container management commands

Pulling an Image:

- Download an public image from a Container registry

Pulling an Image with specific version:

- Download a specific version of a public image from a Container registry

Pulling an Private Image from Registry:

- To pull a private Docker image from a registry, you need to follow these steps:

Docker Login: Authenticate with the registry using the `docker login` command (Default Registry: Docker Hub)

Pull the Image: Use the `docker pull` command with the full image name (including the registry URL)

Example:

Running an Image:

- Runs a container in detached mode (container runs in the background) and maps a host port to a container port
- Runs a container interactively (you can execute commands) with a terminal session

Listing:

- List Currently Running Containers
- List All Containers (Running + Stopped)
- Legacy Equivalent Commands (Still Commonly Used)

Start:

- Starts a stopped container

Stop:

- Stops a running container by sending a SIGTERM signal, followed by a SIGKILL signal after a grace period, terminating all processes in the container

A SIGTERM signal is a request for a process to terminate gracefully

While a SIGKILL signal is a forceful and immediate termination

Kill:

- Immediately stops a container by sending a SIGKILL signal

Restart:

- Restarts a running container

Pause:

- Pauses all processes within the container, freezing them in their current state

Unpause:

- Unpauses all processes, allowing the container to continue from where it was paused

Copy Files from Host to Running Container:

- Copies a file or directory from your host machine into a running container

Remove:

- Removes one or more stopped containers
- Removes all stopped containers

View logs

- Fetches the logs of a container
- Fetches the logs of a container and follows the log output

Managing Images:

- Lists all images on the local machine
- Displays the history of an image, showing each layer that was created during the build process

Remove:

- Delete one or more Docker images from the local system

Cleanup:

- Removes unused data such as containers, images, networks, and build cache

Dockerfile

```
docker pull <IMAGE_NAME>
```

Dockerfile

```
# Version or Tag
docker pull <IMAGE_NAME>:<SPECIFIC_VERSION>

# Example:
docker pull node:24.01
```

Dockerfile

```
# <registry-name>.azurecr.io
docker login <registry-url>
```

Dockerfile

```
docker pull <registry-url>/<image-name>:<tag>
```

Dockerfile

```
docker login myreg.example.com  
docker pull myreg.example.com/my-private-image:latest
```

Dockerfile

```
# <registry-name>.azurecr.io  
docker login <registry-url>
```

Dockerfile

```
docker pull <registry-url>/<image-name>:<tag>
```

Dockerfile

```
docker login myreg.example.com  
docker pull myreg.example.com/my-private-image:latest
```

Dockerfile

```
# HOST_PORT: The port number on your host machine  
#           where you want to receive traffic  
# CONTAINER_PORT: The port number within the container  
#                 that's listening for connections  
docker run -d -p <HOST_PORT>:<CONTAINER_PORT> <IMAGE_NAME>
```

Dockerfile

```
docker run -it <IMAGE_NAME> /bin/bash
```

Dockerfile

```
docker container ls
```

Code

CONTAINER ID	IMAGE	STATUS	NAMES
d1f4e5c8a123	nginx:latest	Up 5 minutes	sharp_haibt
f3c9e7a4b456	redis:alpine	Up 2 hours	eloquent_mo
abc1234def56	mysql:8.0	Up 24 hours	brave_banac

Dockerfile

```
docker container ls -a
```

Code

CONTAINER ID	IMAGE	STATUS	NAMES
d1f4e5c8a123	nginx:latest	Up 5 minutes	sharp_haibt
f3c9e7a4b456	redis:alpine	Exited 10 minutes ago	eloquent_mo
abc1234def56	mysql:8.0	Up 24 hours	brave_banac
c4d7e2f9e789	alpine	Exited 2 hours ago	clever_poin

Dockerfile

```
docker ps
docker ps -a
```

Dockerfile

```
docker container start <CONTAINER_ID or NAME>
```

Dockerfile

```
docker container stop <CONTAINER_ID or NAME>
```

Dockerfile

```
docker container kill <CONTAINER_ID or NAME>
```

Dockerfile

```
docker container restart <CONTAINER_ID or NAME>
```

Dockerfile

```
docker container pause <CONTAINER_ID or NAME>
```

Dockerfile


```
docker container unpause <CONTAINER_ID or NAME>
```

Dockerfile

```
docker cp /path/on/host <CONTAINER_ID_or_NAME>:/path/in/container
```

Example:

```
docker cp ./app/config.json my_container:/usr/src/app/config.json
```

Dockerfile

```
docker container rm <CONTAINER_ID or NAME>
```

Dockerfile

```
docker container prune
```

Dockerfile

```
docker logs <CONTAINER_ID or NAME>
```

Code

```
2045-06-30T12:00:00Z Starting Nginx server...
2045-06-30T12:00:01Z Listening on port 80
2045-06-30T12:02:15Z GET /index.html 200 OK
2045-06-30T12:03:44Z GET /favicon.ico 404 Not Found
2045-06-30T12:05:10Z Received SIGTERM, shutting down...
```

Dockerfile

```
docker logs -f <CONTAINER_ID or NAME>
```

Dockerfile

```
docker images
```

Code

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	a1b2c3d4e5f6	2 days ago	142MB
redis	alpine	f6e5d4c3b2a1	1 week ago	32MB
mysql	8.0	1234abcd5678	3 weeks ago	495MB

Dockerfile

```
docker history <IMAGE_ID>
```

Code

```
##simplified
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
a1b2c3d4e5f6   2 days ago       CMD ["nginx"..     0B        -
7f8e9a0b1c2d   2 days ago       EXPOSE 80          0B        -
3c4d5e6f7g8h   2 days ago       RUN apt-get ..     53MB      -
1a2b3c4d5e6f   2 days ago       ADD files ..       22MB      -
```

Dockerfile

```
docker rmi <IMAGE_ID or NAME>
```

Dockerfile

```
docker system prune
```

Q3

How do you set memory & CPU usage for containers & get Memory, and CPU usage info about specific containers?

- To limit the memory usage:
use the --memory (or -m) flag
- To limit the CPU usage:
use the --cpus flag
- Get Usage for a Specific Container:

Dockerfile

```
docker run -d --memory="512m" --cpus="0.5" nginx:latest

# cpus - This restricts the container to use
#         at most 50% of a single CPU core.
# It doesn't mean the container gets a dedicated half-core
# It means the container will not exceed 0.5 CPU
# in terms of total CPU time across available cores.
```

Dockerfile

```
docker stats CONTAINER_ID_or_NAME
```

Code

```
##simplified
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT
d1f4e5c8a123   sharp_haibt  0.45%    25.3MiB / 512MiB
```

Q4

Explain the Docker Container Lifecycle

Create:

- Define a container from an image without starting it

Start:

- Run a container that has been created or restart a stopped container

Run:

- Create and start a container in one step

Stop:

- Gracefully halt the processes in a running container

Kill:

- Forcibly stop a running container immediately

Pause:

- Suspend all processes in a running container

Unpause:

- Resume all processes in a paused container

Restart:

- Stop and then start a container again

Remove:

- Delete a stopped container from the Docker host

Dockerfile

```
docker create --name mycontainer myimage
```

Dockerfile

```
docker start mycontainer
```

Dockerfile

```
docker run --name mycontainer myimage
```

Dockerfile

```
docker stop mycontainer
```

Dockerfile

```
docker kill mycontainer
```

Dockerfile

```
docker pause mycontainer
```

Dockerfile

```
docker unpause mycontainer
```

Dockerfile

```
docker restart mycontainer
```

Dockerfile

```
docker rm mycontainer
```

Q5

How do you get all the system-level information about docker? Like events, disk space, caching, etc..

- Display system-wide information about Docker installation, including number of containers, images, and storage driver details
 - Show Docker disk usage, including total and used space for images, containers, and volumes
 - Monitor Docker events in real-time, such as container starts/stops, image pulls, and network changes
 - Display real-time CPU, memory, and network usage statistics for running containers
 - Show the history of an image, including commands and metadata used to create each layer
 - Return low-level information about Docker objects, such as containers, images, networks, and volumes
 - Remove unused data, such as stopped containers, dangling images, and unused networks and volumes
 - Remove unused images
 - Remove unused containers
 - Remove unused networks

Dockerfile

```
docker info
```

Code

```
Server Version:      34.0.2
Storage Driver:      overlay2
Number of Containers: 5 (3 running, 2 stopped)
Number of Images:    12
Default Runtime:     runc
```

Dockerfile

```
docker system df
```

Code

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	12	4	2.4GB	1.5GB (62%)
Containers	5	2	450MB	300MB (67%)
Local Volumes	3	2	600MB	150MB (25%)
Build Cache	15	-	1.2GB	900MB (75%)

Dockerfile

```
docker events
```

Code

```
2045-06-30T12:34:56Z container create
d1f4e5c8a123 (image=nginx:latest, name=sharp_haibt)

2045-06-30T12:34:57Z container start
d1f4e5c8a123 (image=nginx:latest, name=sharp_haibt)

2045-06-30T12:36:20Z image pull
sha256:def456 (nginx:latest)

2045-06-30T12:38:42Z network connect
container=abc123 network=bridge
```

Dockerfile

```
docker stats [container_id]
```

Code

```
##simplified
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT
d1f4e5c8a123   sharp_haibt 0.45%     25.3MiB / 512MiB
```

Dockerfile

```
docker history [image_name]
```

Code

```
##simplified
IMAGE          CREATED        CREATED BY          SIZE
a1b2c3d4e5f6   2 days ago    CMD ["nginx"..     0B
7f8e9a0b1c2d   2 days ago    EXPOSE 80          0B
3c4d5e6f7g8h   2 days ago    RUN apt-get ..     53MB
1a2b3c4d5e6f   2 days ago    ADD files ..       22MB
```

Dockerfile

```
docker inspect [object_name]
```

Code

```
[
  {
    "Id": "abc1234def5678gh9012ijkl3456mnop7890qrstuvwxyz",
    "Name": "/web_server",
    "Image": "nginx:latest",
    "State": {
      "Status": "running",
      "Running": true,
      "StartedAt": "2045-06-30T12:34:56.789012345Z"
    },
    "NetworkSettings": {
      "IPAddress": "172.17.0.2",
      "Ports": {
        "80/tcp": [
          {
            "HostIp": "0.0.0.0",
            "HostPort": "8080"
          }
        ]
      }
    },
    "Mounts": [
      {
        "Type": "volume",
        "Name": "myvolume",
        "Destination": "/usr/share/nginx/html"
      }
    ]
  }
]
```

Dockerfile

docker system prune

Dockerfile

docker image prune

Dockerfile

docker container prune

Dockerfile

docker network prune

CHAPTER 4

Persisting Data in Docker

8 Questions & Answers

Q1

Why is Data Persistence needed for Containers?

- Ephemeral Nature of Containers: Containers are designed to be short-lived. Data inside a container is lost when it restarts or terminates
 - Stateful Applications Need Persistence: Applications like databases, file storage systems, and message brokers need to retain their state across restarts or crashes
 - Data Retention: Critical data—such as user sessions, configurations, and transactions—must survive beyond the lifecycle of individual containers
 - Backup and Recovery: Persistent storage enables backup strategies and ensures data can be recovered after failure
 - Example Usecases:
 - Running MySQL, PostgreSQL, or MongoDB in a container
 - Web app that allows users to upload profile pictures or documents
 - Shopping cart in an e-commerce containerized frontend
 - Microservice logging activity data

Q2

What is a Volume in Docker?

Volumes:

- Ephemeral Nature of Containers: Containers are designed to be short-lived. Data inside a container is lost when it restarts or terminates
 - Volumes provide persistent storage: Data survives container restarts and removals
 - Volumes enable Data sharing: Multiple containers can access the same volume, enabling shared data

Types of Volumes in Docker:

- 1) Named Volumes: Have a specific name assigned to them when created Example:

Command: Created using command `docker volume create <volume-name>` or automatically when a container specifies a volume that Docker hasn't created yet

- 2) Anonymous Volumes: Created and managed by Docker and don't have a user-assigned name Example:

Usage: Temporary or throwaway data during container runtime.

Used when volume re-usability is NOT important.

Command: Automatically created when a container specifies a volume destination without explicitly creating a named volume

Dockerfile

```
docker volume create volume_name #OPTIONAL!
docker run -d -v volume_name:/path/in/container my_image:tag
```

Dockerfile

```
docker volume create pg_data #OPTIONAL

# Run container
# Docker-managed named volumes (like pg_data)
# are stored on the host filesystem
# but in a location managed internally by Docker
docker run -d \
--name my_postgres \
-e POSTGRES_USER=myuser \
-e POSTGRES_PASSWORD=mypassword \
-e POSTGRES_DB=mydatabase \
-v pg_data:/var/lib/postgresql/data \
-p 5432:5432 \
postgres:latest

# To check if the container is running
docker container ls

# To see the created volume
docker volume ls

# To inspect the volume
docker volume inspect pg_data
# `{"Name": "pg_data",
#   "Mountpoint": "/var/lib/docker/volumes/pg_data/_data", ...}`
```

Dockerfile

```
docker run -d -v /path/in/container my_image:tag
```

Dockerfile

```
# Run container
docker run -d \
--name my_nginx \
-v /var/log/nginx \
-p 8080:80 \
nginx:latest

# To check if the container is running
docker ps

# To see the created anonymous volume
docker volume ls

# To inspect the volume
docker volume inspect VOLUME_NAME # AUTO GENERATED NAME
```

Dockerfile

```
docker volume ls
```

Code

DRIVER	VOLUME NAME
local	9f4b1e3e5a9c4f7d88d46bb5efde14a3
local	project_data_volume
local	812dcff9c2a83c3b8f3dd70d7c0a9b0d
local	mysql_db_data

Q3

Explain Commands used to manage Volumes

Create:

- Create a Named Volume

Lists:

- Lists all Docker volumes

Inspection:

- Returns detailed information about a Docker volume in JSON format

Remove a Specific Volume:

- Deletes a specified Docker volume

Remove All Unused Volumes:

- Removes all volumes that are not currently used by any container

Dockerfile

```
docker volume create project_data_volume
```

Dockerfile

```
docker volume ls
```

Code

```
DRIVER      VOLUME NAME
local       9f4b1e3e5a9c4f7d88d46bb5efde14a3
local       project_data_volume
local       812dcff9c2a83c3b8f3dd70d7c0a9b0d
local       mysql_db_data
```

Dockerfile

```
docker volume inspect <VOLUME_NAME>
```

Code

```
[
  {
    "CreatedAt": "2025-12-01T14:23:45Z",
    "Driver": "local",
    "Labels": {
      "project": "sample-app",
      "env": "dev"
    },
    //Actual path where volume data resides on the host.
    "Mountpoint": "/var/lib/docker/volumes/project_data_volume/_data",
    //The volume's name (UUID for anonymous).
    "Name": "project_data_volume",
    "Options": {},
    "Scope": "local"
  }
]
```

Dockerfile

```
docker volume rm <VOLUME_NAME>
```

Dockerfile

Q4

Explain Bind Mounts with an example

- Bind mounts: Allow us to mount a directory or file on the host machine into a container
- Path linking: Links to a specific path on the host filesystem
- Flexibility: Offers direct access to host files, ideal for development and local testing
- Use case: Real-time data sharing between host and container
- Command:

Example:

- Web Development: Developing a website locally using Docker containers and website files are stored on your host machine at /tmp/website & any changes made on your host to be instantly visible in the container running your web server
- Command:

Dockerfile

```
docker run -d -v /host/path:/container/path my_image:tag
```

Dockerfile

```
mkdir /tmp/website

# Run the Apache Container
docker run -d --name my_apache \
-v /tmp/website:/usr/local/apache2/htdocs \
-p 8080:80 \
httpd:latest

# Check if Apache is serving the page
curl http://localhost:8080

# Edit the index.html file on the host
echo "<h1>Apache Server Updated - Live Changes</h1>" > \
/tmp/website/index.html

# Verify Changes with curl
curl http://localhost:8080

# Verify Inside the Container
docker exec -it my_apache /bin/sh

# Try to read the index.html file
cat /usr/local/apache2/htdocs/index.html
```

Q5**Docker Volumes vs Bind Mounts**

Feature / Use Case	Docker Volumes
Definition	Managed by Docker: Stored in Docker's internal storage (usually under <code>/var/lib/docker/volumes/</code>).
Management	Managed by Docker
Portability & Isolation	Portable and isolated from host filesystem. Easy to backup, inspect, and share across containers.
Use Cases	Databases (e.g., MySQL, Postgres) storing data.

Q6**Is it possible to share data between Multiple Containers?**

- Yes!!: Possible to share data between multiple containers in Docker.

1) Data Sharing Using Volumes:

- Recommended method: Docker volumes are the preferred way to share data between containers
 - Named volumes: Create and mount a named volume into both containers
 - Shared data store: Both containers can read from and write to the same volume
 - Example:

2) Data Sharing Using Bind Mount:

- Bind mounts: Can be used to share data between containers by mounting the same host directory into both containers
 - Example:

3) Data Sharing Using Volumes from Containers:

- Volume sharing: One container can access volumes from another container
If the container has multiple volume mounts, --volumes-from inherits all of them
- Data sharing: Useful for sharing data between running containers
- Example

Dockerfile

```
# Create a named volume
docker volume create shared_data
```

Dockerfile

```
# Container 1 mounts the volume
docker run -d --name container1 -v shared_data:/data my_image1
```

Dockerfile

```
# Container 2 mounts the same volume
docker run -d --name container2 -v shared_data:/data my_image2
```

Dockerfile

```
# Create a named volume
docker volume create shared_data

# Run the first container (Producer)
docker run -d --name container1 \
-v shared_data:/data alpine sh -c \
"echo 'Hello from container1' > /data/hello.txt && tail -f /dev/null"

# Run the second container (Consumer)
docker run --name container2 \
-v shared_data:/data alpine cat /data/hello.txt && sleep 5s
```

Dockerfile

```
# Container 1 mounts a host directory
docker run -d --name container1 -v /host/path:/data my_image1
```

Dockerfile

```
# Container 2 mounts the same host directory
docker run -d --name container2 -v /host/path:/data my_image2
```

Dockerfile

```
# Create a Host Directory for Shared Data
mkdir -p /tmp/shared_logs && cd /tmp/shared_logs

# Start the First Container (Producer)
docker run -d --name container1 \
-v /tmp/shared_logs:/data alpine sh -c \
"while true; do echo $(date) >> /data/log.txt; sleep 1; done"

# Start the Second Container (Consumer)
docker run --rm --name container2 \
-v /tmp/shared_logs:/data alpine sh -c "tail -f /data/log.txt"
```

Dockerfile

```
# Start container 1 with a named volume
docker run -d --name container1 \
-v shared_data:/data my_image1
```

Dockerfile

```
# Start container 2 and use the volume from container 1
docker run -d --name container2 \
--volumes-from container1 my_image2
```

Dockerfile

```
# Start Container 1 (Uploader/Producer)
docker run -d --name container1 \
-v shared_data:/data alpine sh -c \
"echo 'Hello from container1' > /data/data.txt && tail -f /dev/null"

# Start Container 2 (Processor/Consumer)
docker run --rm --name container2 \
--volumes-from container1 alpine cat /data/data.txt
```

Q7

How can you recover data after accidentally deleting a Container?

- Volume Storage: Data stored in a Docker volume remains intact even after container removal, allowing reuse with new containers. `docker run -v my_volume:/app/data my-image`
- Bind Mounts from Host: Data saved in a host directory (e.g., `-v $(pwd)/data:/app/data`) persists independently of the container. `docker run -v /your/local/path/data:/app/data my-image`
- Container Filesystem: Data stored inside the container's filesystem is lost once the container is deleted, and cannot be recovered unless backed up.
- Recommendations:
 - Always use volumes or bind mounts for important data to prevent data loss.
 - Regularly back up data stored within containers if not using volumes or bind mounts.

- **Default Storage:** Docker uses the local filesystem to store data by default
- **Volume Drivers:** Plugins that enable Docker to create and manage volumes with different backends
 - **External Storage Management:** Support for storing, managing, and accessing data outside the local system
 - Data Sharing Across Hosts: Use of systems like NFS to share data between multiple Docker hosts
 - Cloud Storage Integration: Compatibility with cloud services such as Amazon EBS or Azure File Share
 - **Additional Capabilities:** Enable additional capabilities
 - Enhanced Redundancy: Improve data reliability and availability through specialized storage solutions
 - Performance Optimization: Different drivers can optimize performance based on backend storage capabilities
 - Security Features: Some volume drivers support encryption and access controls for sensitive data
 - Cross-Platform Compatibility: Drivers can facilitate data access across various operating environments
- **Custom Volume Creation:** Use command like `docker volume create --driver DRIVER_NAME --opt key=value volume_name` to specify drivers and options

CHAPTER 5

Docker Networking

7 Questions & Answers

Q1

Why do we need Container Networking?

- Container networking: Essential for enabling communication between containers, the host, and external systems.
- Features: Allows containers to:
 - Share data and APIs with other services
 - Be isolated or exposed securely
 - Be part of multi-container apps (e.g., microservices)
- Default bridge network: When you install Docker Desktop (for example), a default bridge network (also called bridge) is created automatically
 - Default for New Containers: Newly-started containers connect to it unless otherwise specified
 - Default outgoing connections allowed: Containers can make outgoing connections
 - Incoming connections from host denied by default: You cannot access anything on the containers without publishing the ports to the host

Q2

What are the Networking Options in Docker?

Bridge:

- Default Network: When you start Docker, a default bridge network (also called bridge) is created automatically, and newly-started containers connect to it unless otherwise specified.
- Create Custom Networks: You can also create your own custom bridge networks and connect containers to them.
- Docker embedded DNS: Containers on the same bridge network can communicate using container names as host names(Simplifies service discovery without manual IP management)
- Provides Network Isolation: Offer network isolation by default: containers on one bridge network cannot access containers on another bridge unless explicitly connected
- Port mapping Needed: Port mapping (e.g., -p 8080:80) is required to expose container services to the host

Host:

- **Shares Host Network Stack:** In host networking mode, the container shares the host machine's network stack directly (no virtual network bridge is created) and the container uses the host's IP address
 - **No Network Isolation:** Since there's no network translation or NAT involved, host networking offers lower latency and higher throughput, making it suitable for performance-sensitive applications
 - **Only one Host Network allowed:** host is a special, built-in network that directly maps the container's network stack to the host — and only one such network can exist.
 - **Usecase:** Best for apps needing direct access to the host's network for maximum performance
 - **Constraint:** Containers using host networking cannot bind to the same ports as other host services or containers using the same mode, leading to potential port conflicts

None:

- **No Network Connectivity:** Containers started with the none network driver have no network connectivity — not even to other containers or the host.
- **Complete Isolation:** Provides complete network isolation for containers
- **Most isolated network mode:** Most isolated network mode in Docker and is typically used in advanced scenarios like security testing etc.

Overlay:

- **Multiple Docker Hosts (Docker Swarm):** Overlay networks enable containers running on different Docker hosts to communicate securely, as if they were on the same network
- **Built-in DNS:** Provides built-in DNS for service name resolution, making cross-host container communication seamless

Macvlan:

- **Unique IP Addr For Each Container:** Macvlan gives each Docker container its own IP address, so it behaves like a separate computer on your network—not just a sub-process inside your machine.
- **Usecase:** You're containerizing a legacy application (e.g., an old monitoring system or database tool) that expects to:
 - Have its own static IP address,
 - Be directly accessible on the local network and
 - Communicate with other physical machines or legacy systems that don't support Docker-aware networking.

Dockerfile

```
$ docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
a1b2c3d4e5f6    bridge    bridge      local
b2c3d4e5f6g7    host      host         local
c3d4e5f6g7h8    none      null         local
```

Dockerfile

```
docker network create --driver bridge my_bridge_network

# Assigning two containers to the bridge network
# These container can talk with each other using their names
# From container2, you can access a web app on port 8080
# using (http://container1:8080)
docker run -dit --name container1 \
  --network my_bridge_network alpine sh
docker run -dit --name container2 \
  --network my_bridge_network alpine sh
```

Dockerfile

```
# HOST_PORT: The port number on your host machine
#           where you want to receive traffic
# CONTAINER_PORT: The port number within the container
#               that's listening for connections
docker run -d -p <HOST_PORT>:<CONTAINER_PORT> <IMAGE_NAME>
```

Dockerfile

```
docker run -dit --name container1 --network host alpine sh
```

Dockerfile

```
docker network create --driver overlay my_overlay
# Note: Only works if you're using docker swarm
```

Dockerfile

```
docker network create --driver macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 \
  macvlan_net
```

Q3

Give Examples of Network Management Commands in Docker?

Create:

- Create a custom network

List:

- List all Docker networks

Inspection:

- Returns detailed information about a Docker network in JSON format

Remove:

- Removes a custom network (only if unused)

Dockerfile

```
docker network create --driver bridge my_custom_network
docker network create --driver overlay my_overlay_net
```

Dockerfile

```
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d9b100f2d636        bridge             bridge             local
e2fb1c7bcd1d        host              host              local
7a6ef9c3c6c2        none              null              local
ae34f1b2ab01        my_overlay_net     overlay           swarm
f4a678e3cf79        my_bridge_net      bridge            local
```

Dockerfile

```
$ docker network inspect <NETWORK_ID or NAME>
[
  {
    "Name": "my_bridge_net",
    "Id": "f4a678e3cf79492b7e3d12c5f",
    "Driver": "bridge",
    "Scope": "local",
    "IPAM": {
      "Driver": "default",
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Containers": {
      "bca1c0b45357bd5a...": {
        "Name": "container1",
        "IPv4Address": "172.18.0.2/16",
        "MacAddress": "02:42:ac:12:00:02"
      },
      "58f0e8eafadfbe29...": {
        "Name": "container2",
        "IPv4Address": "172.18.0.3/16",
        "MacAddress": "02:42:ac:12:00:03"
      }
    },
    "Options": {},
    "Labels": {}
  }
]
```

Dockerfile

```
docker network rm <name>
```

Q4

How can containers be isolated using a custom Bridge Network?

- Create a Custom Bridge Network:
- Run Containers in the Custom Network:
- Communication Between Containers: Containers connected to the same custom bridge network (isolated-network in this case) can communicate with each other using their container names

But containers outside the isolated-network will not be able to talk to container1 and container2

Dockerfile

```
docker network create isolated-network
```

Dockerfile

```
docker run -d --name container1 \
--network isolated-network nginx

docker run -d --name container2 \
--network isolated-network mysql
```

Q5

Can a container be connected to multiple networks? If yes, how?

- Yes, a Docker container can be connected to multiple networks.
- When you create a container, you assign it to one network using `--network`.
- You can then connect it to additional networks using `docker network connect`

Dockerfile

```
docker run -dit --name myapp --network net1 alpine
docker network connect net2 myapp
```

Q6

What is DNS-based service discovery in Docker networks?

- DNS-based service discovery: Ability for containers to resolve each other's names to IP addresses using an embedded DNS server provided by Docker.
- When you create a user-defined bridge or overlay network, Docker automatically sets up an internal DNS server.
- Every container attached to that network is registered by name.
- Other containers in the same network can communicate using that name, without needing IPs or environment variables.

Dockerfile

```
docker network create my_network
docker run -dit --name web_container --network my_network nginx
docker run -dit --name app_container --network my_network alpine sh

#Inside app_container, ping web_container:
docker exec -it app_container sh
ping web_container
# This works because Docker's internal DNS
# resolves web_container to its container IP.
```

Q7

What happens when two containers are in different networks? How can you enable communication between them?

- When two containers are placed in different Docker networks, they are logically isolated from each other:

They cannot ping, connect, or resolve each other's names.

Each network has its own IP range and DNS namespace.

This isolation improves security.

Option 1: Connect to Same Network

Option 2: Publish a Port on Host

Other Options

- Overlay Network on Multiple Hosts
- Macvlan Network
- ...

Dockerfile

```
docker network create netA
docker network create netB

docker run -dit --name containerA --network netA alpine
docker run -dit --name containerB --network netB alpine

# Create a common network - netCommon
docker network create netCommon
docker network connect netCommon containerA
docker network connect netCommon containerB

# OR Connect containerB to netA
# docker network connect netA containerB
```

Dockerfile

```
docker run -d --name containerA \
  --network netA \
  -p 5000:5000 \
  my-web-app

docker run -dit --name containerB --network netB alpine

docker exec -it containerB sh
curl http://HOST_IP:5000
```

CHAPTER 6

Docker Security

9 Questions & Answers

Q1

Why is Container Security Important?

- **Shared OS Kernel:** Containers share the host OS kernel, so vulnerabilities here can affect all containers.
- **Use of Base Images:** Untrusted or tampered images can include malicious code, so it's important to use signed images, verify checksums, and pull from trusted sources.
- **Use of Third-party Dependencies:** Containers often include external libraries which may have security issues if not regularly scanned and updated.
- **Fast Lifecycle:** Containers are short-lived and rapidly deployed. Security must be integrated into the build and deployment process (DevSecOps).
- **Platform Security:** Containers must meet compliance requirements (e.g., PCI DSS, HIPAA, GDPR). Implement logging, access control, and auditing from the ground up.

Q2

What are the Best Practices with Securing Containers?

- **Use Official Images:** Use official Container images from trusted sources
 - **Regular Updates and Patch Management:** Keep Docker Engine and container images up to date with security patches
 - **Minimize Attack Surface:** Use minimal base images (e.g., Alpine), and remove unnecessary packages/tools
 - **Image Scanning:** Scan images using tools like Docker Scout, Trivy, or Clair to detect vulnerabilities early
 - **Run as Non-Root User:** Never run containers as root—define a dedicated user in the Dockerfile
 - **Principle of Least Privilege:** Grant only the minimum permissions required to the container and its processes
 - **Limit Linux Capabilities:** Drop unused capabilities using the `--cap-drop` flag for tighter control
 - **Avoid Hardcoding Secrets:** Do not store passwords, tokens, or API keys in images or environment variables
 - **Secrets Management:** Use Docker Secrets, HashiCorp Vault, or AWS Secrets Manager to handle credentials securely
 - **Network Segmentation:** Use Docker's network features (e.g., bridge, overlay) to isolate containers and limit lateral movement
 - **Restrict External Access:** Only expose necessary ports and endpoints
 - **Container Hardening:** Configure and enforce security policies (e.g., Docker Bench Security)
- Key practices include restricting container capabilities, enforcing user namespaces, and reducing default privileges.
- **Observability:** Implement monitoring, logging and tracing to detect and respond to security incidents promptly
- Collect and centralize container logs using tools like Fluentd, ELK, or Datadog
- Use tools like Jaeger or Grafana Tempo to trace and troubleshoot security incidents

Q3

How can you run a Container as Non-Root User?

- **Avoid root user:** Limit security risks by not running containers as root
- **Create group:** Set up a system group named `appgroup`
- **Create user:** Add a system user named `appuser`
- **Assign user to group:** Place `appuser` in the `appgroup`
- **Set home directory:** Assign `/usr/src/app` as the home directory for `appuser`

Dockerfile

```

FROM node:14

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install

COPY . .

# Create a non-root user and group
RUN groupadd -r appgroup && useradd -r -g appgroup -d /usr/src/app appuser

# Change ownership of the app directory
RUN chown -R appuser:appgroup /usr/src/app

# Switch to the non-root user
USER appuser

EXPOSE 3000

CMD ["node", "app.js"]

```

Q4

How do you remove Linux capabilities from a container?

- Examples: Example Linux Capabilities:
 - CAP_CHOWN: Change file ownership
 - CAP_SETUID: Change user ID
 - CAP_NET_BIND_SERVICE: Bind to ports < 1024 (e.g., 80, 443). Needed by web servers like NGINX or Apache
 - CAP_NET_RAW: Use raw sockets
 - CAP_SYS_TIME: Set the system clock
 - CAP_SYS_ADMIN: Powerful and wide-ranging, includes mounting filesystems, chroot, etc.
 - CAP_AUDIT_WRITE: Write records to the kernel audit log
- NOT all capabilities are needed: Typical applications do NOT need all capabilities
 - Recommended: Selectively add the capabilities that are needed
- **--cap-drop**: Use Docker option **--cap-drop** to remove Linux capabilities from a container
 - Drop all capabilities: **--cap-drop=ALL** removes all privileged operations
 - Add specific capability: **--cap-add=NET_ADMIN** allows network administrative tasks
- Example command:
 - Always start with **--cap-drop=ALL** and selectively add only what's absolutely necessary using **--cap-add**.
 - Review application needs carefully before allowing elevated capabilities.

Dockerfile

```
docker run --cap-drop=ALL --cap-add=NET_ADMIN myimage
```

- **Ensure Smooth Container Operations:** Health check is crucial for maintaining the reliability and stability of your container
 - **HEALTHCHECK Instruction:** Add a health check in the Dockerfile to periodically test if the container is healthy Example:
 - **CMD replace_with_your_command:** The command to run for the health check. It should return a zero exit code if healthy, or a non-zero code if unhealthy

CMD replace_with_your_command || exit 1 is a common shell scripting pattern - Run this command. If it fails (i.e., returns a non-zero exit code), then exit the script immediately with exit code 1.
 - **--interval=30s:** Sets the time between consecutive health checks

The container will perform a health check every 30 seconds
 - **--timeout=10s:** Specifies the maximum time allowed for the health check command to complete

If the health check doesn't respond within 10 seconds, it's considered a failure
 - **--retries=3:** Determines the number of consecutive failures needed to consider the container as unhealthy

After 3 failed health checks, Docker marks the container as unhealthy
 - **--start-period=5s:** Provides a grace period after container start before initiating health checks

Docker waits for 5 seconds after the container starts before performing the first health check

Below are the three primary methods to check the health of a container

- HTTP Endpoint
- TCP Port
- Custom Script

HTTP Endpoint: This method involves sending an HTTP request to a specific endpoint within the container to verify if the web server or application is responding correctly

- Example:

TCP Port: This method checks whether a specific TCP port is open and accepting connections, ensuring that the service within the container is listening correctly

- Example:

Custom Script: This method utilizes a custom script to perform more complex or specific health checks tailored to the application's logic within the container

- healthcheck.sh: Custom Script
- Example:

Code

```
# Define the health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 --start-period=5s \
  CMD replace_with_your_command || exit 1
```

Dockerfile

```
# Use an appropriate base image
FROM nginx:latest

# Define the health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD curl --fail http://localhost:80/ || exit 1

# Expose port 80
EXPOSE 80
```

Dockerfile

```
# Use an appropriate base image
FROM postgres:latest

# Define the health check
# nc -z localhost 5432 || exit 1
# check if a service is listening on port 5432
# and to exit with an error if it is not.
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD nc -z localhost 5432 || exit 1

# Expose port 5432
EXPOSE 5432
```

Code

```
#!/bin/bash

# Example health check: Verify if the application process is running
if pgrep httpd > /dev/null; then
  exit 0
else
  exit 1
fi
```

Dockerfile

```
# Use an appropriate base image
FROM httpd:latest

# Copy the custom health check script into the container
COPY healthcheck.sh /usr/local/bin/healthcheck.sh

# Update package lists before installing procs
RUN apt-get update && apt-get install -y procs

# Make the script executable
RUN chmod +x /usr/local/bin/healthcheck.sh

# Define the health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD /usr/local/bin/healthcheck.sh

# Expose necessary ports
EXPOSE 8080
```

Q6

Explain Docker Content Trust (DCT) and how it enhances container security?

- Question: How can the user of an image be certain that the image was not modified after it was published by the image publisher?

Enable Docker Content Trust (DCT)

- Docker Content Trust (DCT): Security feature that lets you verify the integrity & publisher of image

The image is published by a trusted source

The image hasn't been tampered with

- Notary: Tool used by DCT to cryptographically sign and verify container images
- Enabling DCT: Create an Environment Variable `DOCKER_CONTENT_TRUST` with value

1

Only signed images are pulled or pushed (when enabled)

Prevents pulling unsigned images

Prevents pushing images without a valid signature

- Example: Pulling an Image

If the image is signed, it will be pulled.

If not signed, it will fail with: Error: remote trust data does not exist for alpine: notary.docker.io

Verifying Trust Data

Code

```
export DOCKER_CONTENT_TRUST=1
```

Dockerfile


```
export DOCKER_CONTENT_TRUST=1
docker pull alpine:latest
```

Dockerfile

```
$ docker trust inspect --pretty myrepo/myapp
Signatures for myrepo/myapp:

# SIGNED TAG shows which tags are signed (e.g., latest)
SIGNED_TAG      DIGEST      SIGNERS
latest         7b0f2b97d114b57a6b8f8c7e    alice

# SIGNERS are authorized entities who signed the image
List of signers and their keys:

SIGNER          KEYS
alice           03d4dc9e7ad6c8c9e173aa6c80f2b67

Administrative keys for myrepo/myapp:

Repository Key:  6c3253b7a2d49f99d5e0b96b2db3a1ed
Root Key:       2fb4456e0bb79e62f4aa5ce7e2580b31
```

Q7

What are CVEs (Common Vulnerabilities and Exposures)?

- CVE: Publicly disclosed security vulnerability in software or hardware.
- Uniquely Identified: Each CVE is assigned a unique identifier and a brief description of the issue.
 - Standardized Information Sharing: Maintained by the MITRE Corporation and used globally to standardize and share information about cybersecurity threats.
 - Each CVE typically includes:
 - ID: e.g., CVE-2023-45678
 - Description: Brief summary of the vulnerability
 - Affected Products: Software, OS, or libraries impacted
 - References: External advisories or patch info (e.g., NIST, GitHub)
 - CVSS Score: A severity rating from 0 to 10 (see below)
 - 7.0–8.9 (High Impact), 9.0–10.0 (Critical)
 - Why CVEs Matter to Developers & DevOps: Helps you identify vulnerabilities in open-source libraries, Docker images, and IaC
 - Real-World Example: CVE-2021-44228 The Log4Shell vulnerability in Apache Log4j
 - Affected millions of Java applications
 - CVSS Score: 10.0 (Critical)
 - Allowed remote code execution without authentication

Q8

What is Docker Image Vulnerability Scanning?

- **Docker Images can contain vulnerabilities:** Docker images, built in layers, can inherit vulnerabilities from their base images, third-party dependencies, and even misconfigurations in the Dockerfile

Vulnerable Base Images: Using outdated or unpatched base images can expose your application to known exploits

Outdated Dependencies: Libraries and packages within your image that haven't been updated can contain security flaws

Hardcoded Secrets: Storing sensitive information like API keys, passwords, or access tokens directly in the image is a major security risk

Excessive Permissions: Running containers as a root user or with unnecessary capabilities increases the potential impact of a compromise

Unnecessary Components: Including build tools, development libraries, or other components not required for runtime increases the attack surface.

- **Scan to detect early:** Scanning helps detect and mitigate these risks early

Shift Left: Identifying and fixing vulnerabilities early in the development lifecycle (shifting left) prevents them from reaching production environments, where they can be much more costly and disruptive

- **Popular Tools:**

Docker Scout (by Docker Inc.): Docker-native vulnerability scanning. Integrated into Docker Desktop and CLI.

Trivy (Aqua Security): Fast, easy-to-use, and supports scanning of OS packages, application dependencies, ..

Clair (by CoreOS) : Static analysis of vulnerabilities in containers. Open-source project that scans Docker images by comparing installed packages against known vulnerabilities.

Snyk: Scans code, containers, and dependencies for known CVEs (Common Vulnerabilities and Exposures)

Grype: A vulnerability scanner for container images and filesystems

- **Best Practice:**

Scan early and often – Integrate scans into your CI/CD pipelines

Q9

How can you scan Docker images with Trivy?

Trivy is a simple yet powerful vulnerability scanner for containers. It checks both operating system packages and software dependencies (like Python packages or Node.js modules).

Basic Scan: To scan a Docker image, run:

- **Output:** Lists found vulnerabilities (e.g., CVE identifiers, severity levels, and potential fixes)

Fix Vulnerabilities: After running a Trivy scan, you may see a list of vulnerabilities along with recommended fixes

- Upgrade the Base Image: Use a more recent or stable base image that has fewer known vulnerabilities
- Apply Security Patches: Update OS packages
- Update Application Dependencies: Modify requirements.txt, package.json, or equivalent files to use newer, patched versions
- Remove Unnecessary Packages: Remove libraries not needed by your application to reduce the attack surface
- Implement Security Best Practices:
 - Use multi-stage builds to keep the final image lightweight
 - Run as a non-root user to minimize privileges
 - Store secrets outside the image (e.g., Docker Secrets or an external vault)
- Rescan: After making changes, run Trivy again to confirm that vulnerabilities are resolved

Dockerfile

```
trivy image <image_name>

# Example
# Pull an example image
docker pull nginx:latest

# Scan the nginx:latest image
trivy image nginx:latest
```

CHAPTER 7

Docker Compose

6 Questions & Answers

Q1

What is the need for Docker Compose?

Running Individual Containers is Repetitive

- Microservices have complex needs: When developing microservices, we often need to run multiple containers together—e.g., a frontend, backend service, database, and other dependencies.

- Repetitive: Running each container individually using `docker run` is repetitive and error-prone

- Create and manage shared networks, volumes, ..

- Launch individual containers

- Define and maintain environment variables, port mappings, and restart policies

- Set up service dependencies

Example for Two Microservices

Let's consider two services: currency-exchange and currency-conversion.

- currency-exchange: The Currency Exchange Service provides exchange rates for currency pairs—for example, it can tell you that 1 USD equals 60 INR.

- currency-conversion: The Currency Conversion Service uses these rates to calculate conversions (e.g., 10 USD = 600 INR) by calling the Currency Exchange Service and multiplying the result based on the requested quantity.

- You want to launch both the services on your local machine

Explore the Microservices project with detailed guides

Create a Docker Network

Run currency-exchange Container

Run currency-conversion Container

Clean-up Commands

Docker Compose Makes it Simple

- Declarative Approach: Provides a declarative approach to managing multi-container applications.
- Single YAML file with definitions: Define services, networks, volumes, dependencies, and environment variables in one docker-compose.yml file
- Easy Command to Launch: Bring up the entire stack with a single command: docker compose up
- Easy Command to Tear Down: Tear everything down cleanly: docker compose down
- Version Controlled: Maintain a version-controlled setup
 - Ensure Consistency: Ensure consistent, reproducible deployments
 - Easily Shareable: Easily share your setup with teammates

Summary: Without vs With Compose

Task	Manual Docker Commands
Create Network (Or Volume, ..)	docker network create
Start Containers	Multiple docker run
Set Environment Variables	-e flags
Manage Dependencies	Manual order handling
Clean Up	stop, rm, network rm
Maintain Consistency	Error-prone

Dockerfile

```
docker network create currency-compose-network
```

Dockerfile

```
docker run -d \
  --name currency-exchange \
  --network currency-compose-network \
  -p 8000:8000 \
  --restart always \
  in28min/currency-exchange:0.0.1-RELEASE
```

Dockerfile

```
docker run -d \
  --name currency-conversion \
  --network currency-compose-network \
  -p 8100:8100 \
  --restart always \
  -e CURRENCY_EXCHANGE_SERVICE_HOST=http://currency-exchange \
  in28min/currency-conversion:0.0.1-RELEASE
```

Dockerfile

```
docker stop currency-conversion currency-exchange
docker rm currency-conversion currency-exchange
docker network rm currency-compose-network
```

Q2

How is Docker Compose different from Docker Swarm or Kubernetes?

Docker Compose vs Docker Swarm vs Kubernetes

- **Docker Compose:** For local development, testing, and simple setup on a single host.
Easy local multi-container orchestration
Use Case: Developers running microservices on their laptop
- **Docker Swarm:** For lightweight container orchestration across multiple machines
Docker-native clustering/orchestration
Use Case: Small-to-medium-scale production deployments where full Kubernetes is overkill (Very rarely used these days!)
- **Kubernetes:** Enterprise-grade container orchestration at scale.
Multi-cloud and hybrid deployment support
Use Case: Large-scale production environments where reliability, scaling, and flexibility are critical

Comparison Table

Feature / Tool	Docker Compose	Docker Swarm
Deployment Scope	Single host	Multi-host (cluster)
Complexity	Low	Medium
Orchestration Support		
Ecosystem	Minimal	Limited
Learning Curve	Easy	Moderate
Use Case	Local Dev	Lightweight Prod
Community Adoption	High (developers)	Very low

docker-compose.yml

- **docker-compose.yml:** A YAML file where you configure the services that make up your application
- **Run Multiple Containers:** Enables you to run multiple Docker containers as a single application

Example 1: docker-compose.yml

Understand it:

- **services:** Heart of the Docker Compose file. Defines all the containers that will be run.
 - services > container_name > image: Uses the specified docker image from Docker Hub or a local registry.
 - services > container_name > ports: Similar to -p flag
 - services > container_name > restart: always: Ensures container automatically restarts if it crashes or host reboots
 - services > container_name > networks: Connects this service to the shared network so it can be discovered by others (e.g., currency-conversion).
 - services > container_name > depends_on: Tells Docker Compose to start currency-exchange before this service. (Note: doesn't wait for service to be ready—just started.)
 - services > container_name > environment: Sets an env variable so the service knows where to contact the currency-exchange microservice. Uses Docker DNS-based service discovery (i.e., http://currency-exchange).
- **networks:** Defines a custom user-defined bridge network
- **volumes:** Defines a custom user-defined volume

Run it:

Another Example: With MySQL Database

- **What?:** Demonstrates a typical web application and database setup using Docker Compose
 - todo-web-application: A Java-based web app that connects to a MySQL database using environment variables.
 - mysql: A MySQL container that stores todos data and credentials defined via environment variables.

Notes

- depends_on: The depends_on entry ensures that mysql starts before the web application.
- mysql-database-data-volume: Ensures data persistence

Your database files will be retained across container restarts

- Environment Configuration

mysql: Bootstraps DB, user, and password on startup

MYSQL_ROOT_PASSWORD: Sets the root (admin) password for MySQL

MYSQL_USER: Creates a new non-root user (todos-user)

MYSQL_PASSWORD: Password for that non-root user

MYSQL_DATABASE: Creates a database named todos and grants access to the above user

todo-web-application: Read by the app to configure its database connection.

RDS_HOSTNAME: Hostname of the MySQL server (here, the service name mysql, which Docker

Compose resolves automatically via internal DNS)

RDS_PORT: MySQL port (3306 by default)

RDS_DB_NAME: Database name to connect to (todos)

RDS_USERNAME: MySQL username (todos-user)

RDS_PASSWORD: Password used by the app to connect to the DB

Example 2: docker-compose.yml

Code

```
services:
  currency-exchange:
    image: in28min/currency-exchange:0.0.1-RELEASE
    ports:
      - "8000:8000"
    restart: always
    networks:
      - currency-compose-network

  currency-conversion:
    image: in28min/currency-conversion:0.0.1-RELEASE
    ports:
      - "8100:8100"
    restart: always
    environment:
      CURRENCY_EXCHANGE_SERVICE_HOST: http://currency-exchange
    depends_on:
      - currency-exchange
    networks:
      - currency-compose-network

# Networks to be created to facilitate communication between containers
networks:
  currency-compose-network:
```

Dockerfile

```
# Build and Run the Microservices
docker compose up

# Accessing the Microservices
# Currency Exchange API:
curl http://localhost:8000/currency-exchange/from/USD/to/INR

# Currency Conversion API
curl http://localhost:8100/currency-conversion/from/USD/to/INR/quantity/10

# Stopping and Cleaning Up
docker compose down
```

Code

```
services:
  todo-web-application:
    image: in28min/todo-web-application-mysql:0.0.1-SNAPSHOT
    #build:
    #context: .
    #dockerfile: Dockerfile
    ports:
      - "8080:8080"
    restart: always
    depends_on: # Start the depends_on first
      - mysql
    environment:
      RDS_HOSTNAME: mysql
      RDS_PORT: 3306
      RDS_DB_NAME: todos
      RDS_USERNAME: todos-user
      RDS_PASSWORD: dummytodos
    networks:
      - todo-web-application-network

mysql:
  image: mysql:5.7
  ports:
    - "3306:3306"
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: dummypassword
    MYSQL_USER: todos-user
    MYSQL_PASSWORD: dummytodos
    MYSQL_DATABASE: todos
  volumes:
    - mysql-database-data-volume:/var/lib/mysql
  networks:
    - todo-web-application-network

# Volumes
volumes:
  mysql-database-data-volume:

networks:
  todo-web-application-network:
```

Q4

How can you use Docker Compose to launch and tear down applications?

Up:

- Builds, (re)creates, starts, and attaches to containers for a service

Down:

- Stops and removes containers, networks, volumes, and images created by up

Restart specific service:

- Command stops and then starts only the specific service without affecting the other running services

Scale services:

- To scale or run multiple instances of the service

Build:

- Builds or rebuilds services defined in the docker-compose.yml

Start:

- Starts existing containers defined in the docker-compose.yml

Logs:

- View the logs of all services defined in the docker-compose.yml

Dockerfile

```
docker compose up
```

```
# For detach mode (running in background), use the -d (follow) option  
docker compose up -d
```

Dockerfile

```
docker compose down
```

Dockerfile

```
docker compose restart <NAME_OF_SERVICE>
```

Dockerfile

```
docker compose up --scale <NAME_OF_SERVICE>=<NO_OF_INSTANCES>
```

```
# Example:  
docker compose up --scale currency-conversion=3
```

Dockerfile

```
docker compose build
```

Code

```
services:  
  todo-web-application:  
    image: in28min/todo-web-application-mysql:0.0.1-SNAPSHOT  
    #build:  
      #context: .  
      #dockerfile: Dockerfile
```

Dockerfile

```
docker compose start
```

Dockerfile

```
docker compose logs  
  
# For continuous, real-time logs, you can use the -f (follow) option  
docker compose logs -f
```

Q5

What is the use of .env files in Docker Compose?

.env files

- By using .env files, you can separate environment-specific configurations from your main docker-compose.yml
- These files help you manage variables like port numbers, image names, volumes, or environment modes.
- Define and manage environment variables that are used within your docker-compose.yml file
- Docker Compose automatically loads a .env file located in the same directory as your docker-compose.yml

1) Base Configuration (docker-compose.yml):

2) Default .env File (Auto-loaded)

3) Custom Environment File: dev.env

4) Command to Use

Code

```
services:
  web:
    image: ${NGINX_IMAGE:-nginx}
    environment:
      - APP_ENV=${APP_ENV}
    volumes:
      - ${HTML_VOLUME}:/usr/share/nginx/html
    ports:
      - "${NGINX_PORT}:80"
```

Code

```
# Development environment
NGINX_IMAGE=nginx:alpine
NGINX_PORT=8080
```

Code

```
# Development environment
APP_ENV=development
HTML_VOLUME=./html/dev
```

Dockerfile

```
docker compose --env-file dev.env up
```

Q6

Docker Compose FAQ

- Q: What happens if you do not specify a network to connect to?
Container is connected to default bridge network
- Q: Difference between build and image in a service definition
build: Builds image from local Dockerfile
image: Uses an existing image
- Q: How can you pass environment variables to a service in Docker Compose?
Using environment: or env_file: in the YAML
- Q: How can you scale a service using Docker Compose?
docker compose up --scale service_name=3
- Q: Can Docker Compose be used in production?
Yes for small setups. But lacks advanced orchestration features.
Kubernetes is better suited for production-scale deployments
- Q: What is the difference between Docker Compose V1 and V2?
V1: Separate binary, V2: Integrated with Docker CLI
Syntax - V1: docker-compose up, V2: docker compose up
V2 is the future — it replaces the need for a separate docker-compose binary

CHAPTER 8

Advanced Docker Concepts

5 Questions & Answers

Q1

Docker FAQ

- Q: What is the use of EXPOSE 8080 command in Dockerfile?

Documents that the container listens on port 8080, but does NOT publish the port.

You still need to map the port using -p or --publish

- Q: How can you run commands in a container?

This gives you an interactive shell inside the container: `docker exec -it <container-name> /bin/bash`

- Q: How can I build images without using cached layers?

Forces Docker to re-execute all layers without using the cache: `docker build --no-cache -t my-image .`

- Q: How do you remove all stopped containers, unused networks, volumes, and images?

Use it with caution `docker system prune -a`

- Q: How can you profile resource usage (CPU, memory) of containers?

Start with `docker stats`. (For deeper insights, integrate with Prometheus/Grafana)

- Q: Why is my container exiting immediately?

Because Docker containers run a single main process. If that process ends, the container exits.

Main process defined in the CMD or ENTRYPOINT in the Dockerfile

If the main process is NOT a long-running process, the container would exit immediately (Ex: `python script.py`)

Common causes:

Your CMD runs a short-lived script (e.g., `python script.py`)

You're using a minimal base image (like alpine) without an interactive shell

CMD or ENTRYPOINT is misconfigured

Tip: Use `docker run -it ubuntu /bin/bash` to keep the container running and interact with it.

Q2

Give Examples of Minimal Docker Base Images

Image Type	Size	Shell?	Detailed Description
scratch	0 MB	No	An empty base image with nothing at all. You must add everything, including libraries.
busybox	~1 MB	Yes	Minimalist Linux distribution with basic Unix utilities. Often used for simple containers or base tools.
alpine	~5 MB	Yes	Lightweight Linux distro with apk package manager. Very popular for size-conscious apps.
distroless	~10–30 MB	No	Maintained by Google. Contains only application runtime dependencies — NO shell, NO package manager, minimal footprint.
debian-slim	~22 MB	Yes	Slimmed-down version of the standard Debian image.

Q3

How would you troubleshoot a container if it is not running?

- Check for Missing or Incorrect Environment Variables: The container may fail if essential environment variables are not set or incorrectly defined
 - Review the Container's Command or Entrypoint: A misconfigured CMD or ENTRYPOINT can cause the container to exit immediately.
 - Diagnose Volume Mount Issues: Permissions errors or missing volume paths may prevent the container from starting
 - Inspect Container Logs: Inspect logs to identify runtime errors or crash messages:
 - Inspect Container Configuration: Examine environment variables, mounted volumes, and networking settings:
 - Check Real-Time Events: Monitor low-level container lifecycle events for issues during startup
 - Verify Resource Usage and Limits: Ensure the container is not constrained by CPU, memory, or I/O limits
 - Examine Docker Engine Logs: Look for system-level or daemon-related failures affecting container behavior

Dockerfile

```
docker logs <container_id>
```

Dockerfile

```
docker inspect <container_id>
```

Dockerfile

```
docker events --filter container=<container_id> --since 2m
```

Dockerfile

```
docker stats <container_id>
```

Code

```
# works on most modern Linux distributions
# Ubuntu, CentOS, Debian
journalctl -u docker.service
```

Q4

How Do Namespaces and Control Groups Enable Container Isolation?

Namespaces — What You Can See and Access

- **Namespaces:** Namespaces are a Linux kernel feature that provides process-level isolation. This is what makes containers appear as separate environments, even though they share the same kernel.
- **Process View:** Namespaces ensure each container only sees its own processes by isolating process IDs
- **Network Separation:** Namespaces provide each container with its own network interfaces, IP addresses, and ports
- **File System Independence:** Namespaces give containers their own view of file system mount points
- **Hostname and Domain:** Namespaces isolate hostname and domain name settings for each container
- **Inter-process Communication (IPC):** Namespaces control how processes communicate within each container
- **User and Group IDs:** Namespaces set separate user and group identities for security and management

Control Groups (CGroups) — What You Can Use

- **CGroups:** Control Groups allow the kernel to limit, measure, and control resource usage per container

Without CGroups, a single container could consume all system resources and affect other containers or the host.
- **Key Capabilities of CGroups:**
 - CPU Limits — Restrict CPU shares or quota per container
 - Memory Limits — Prevent a container from exhausting system RAM
 - Disk I/O Limits — Control read/write throughput
 - Process Limits — Cap the number of processes (PIDs)
 - Network Bandwidth — Limit container-level bandwidth

Summary

- **Lightweight Virtualization:** Combining Namespaces and CGroups creates isolated environments similar to virtual machines but with less system overhead

Q5

How can you run Containers in the Cloud?

Deployment Method	Description	AWS	Azure
Self-Managed on VMs (IaaS)	Install Docker manually on VMs and run containers directly	EC2 with Docker	Azure Virtual Machines
Serverless Containers	Run containers without managing servers; billed per request or execution	AWS Fargate (via ECS/EKS)	Azure Container Instances (ACI)
Platform as a Service (PaaS)	Deploy by pushing code or images	Elastic Beanstalk (Docker support)	Azure App Service (Containers)
Managed Container Orchestration	Run containers at scale using Kubernetes (or ECS)	Amazon EKS (Kubernetes) Amazon ECS (Fargate or EC2)	Azure Kubernetes Service (AKS)
Container Registry	Store and Manage Your Container Images	ECR (Elastic Container Registry)	ACR (Azure Container Registry)

CHAPTER 9

Docker Cheat Sheet

9 Questions & Answers

Q1

System Commands

Command`docker version``docker system info``docker system df``docker system prune`

Q2

Basic Commands

Command`docker run``docker container ls or docker ps``docker container ls -a or docker ps -a``docker stop CONTAINER_NAME``docker rm CONTAINER_NAME``docker pull IMAGE[:TAG]``docker images``docker build -t IMAGE_NAME[:TAG] .`

Q3

Common Options

Option
-d
-p HOST_PORT:CONTAINER_PORT
-v HOST_PATH:CONTAINER_PATH
--name CONTAINER_NAME
-e KEY=VALUE
--network NETWORK
--restart=always
-it
--rm
--memory="512m"
--cpus="0.5"
--user USER
--workdir DIR
--entrypoint CMD

Q4

Container Lifecycle

Command
<code>docker start CONTAINER_NAME</code>
<code>docker restart CONTAINER_NAME</code>
<code>docker pause CONTAINER_NAME</code>
<code>docker unpause CONTAINER_NAME</code>
<code>docker logs CONTAINER_NAME</code>
<code>docker logs -f CONTAINER_NAME</code>
<code>docker stats</code>
<code>docker inspect CONTAINER_NAME</code>
<code>docker exec -it CONTAINER_NAME /bin/sh</code>
<code>docker exec -it CONTAINER_NAME COMMAND</code>
<code>docker top CONTAINER_NAME</code>
<code>docker container prune</code>
<code>docker rename OLD_NAME NEW_NAME</code>
<code>docker kill CONTAINER_NAME</code>

Q5

Images and Volumes

Command

docker image ls

docker image history IMAGE

docker save IMAGE > file.tar

docker load < file.tar

docker tag SOURCE_IMAGE TARGET_IMAGE

docker commit CONTAINER IMAGE_NAME

docker rmi IMAGE

docker image prune

docker volume create VOLUME_NAME

docker volume ls

docker volume inspect VOLUME_NAME

docker volume rm VOLUME_NAME

docker volume prune

Q6

Network

Command

docker network create NETWORK_NAME

docker network ls

docker network inspect NETWORK

docker network connect NETWORK CONTAINER_NAME

docker network disconnect NETWORK CONTAINER_NAME

docker network prune

Q7

Troubleshooting and Debugging

Command

`docker logs CONTAINER`

`docker inspect CONTAINER`

`docker events`

`docker stats`

`journalctl -u docker.service`

Q8

Docker Compose

Command

`docker compose config`

`docker compose up`

`docker compose up -d`

`docker compose up -d --scale SERVICE=NUM`

`docker compose logs`

`docker compose logs -f`

`docker compose ps`

`docker compose exec SERVICE COMMAND`

`docker compose build`

`docker compose restart`

`docker compose down`

Q9

Docker Hub

Command
docker login
docker logout
docker search TERM
docker push IMAGE_NAME[:TAG]

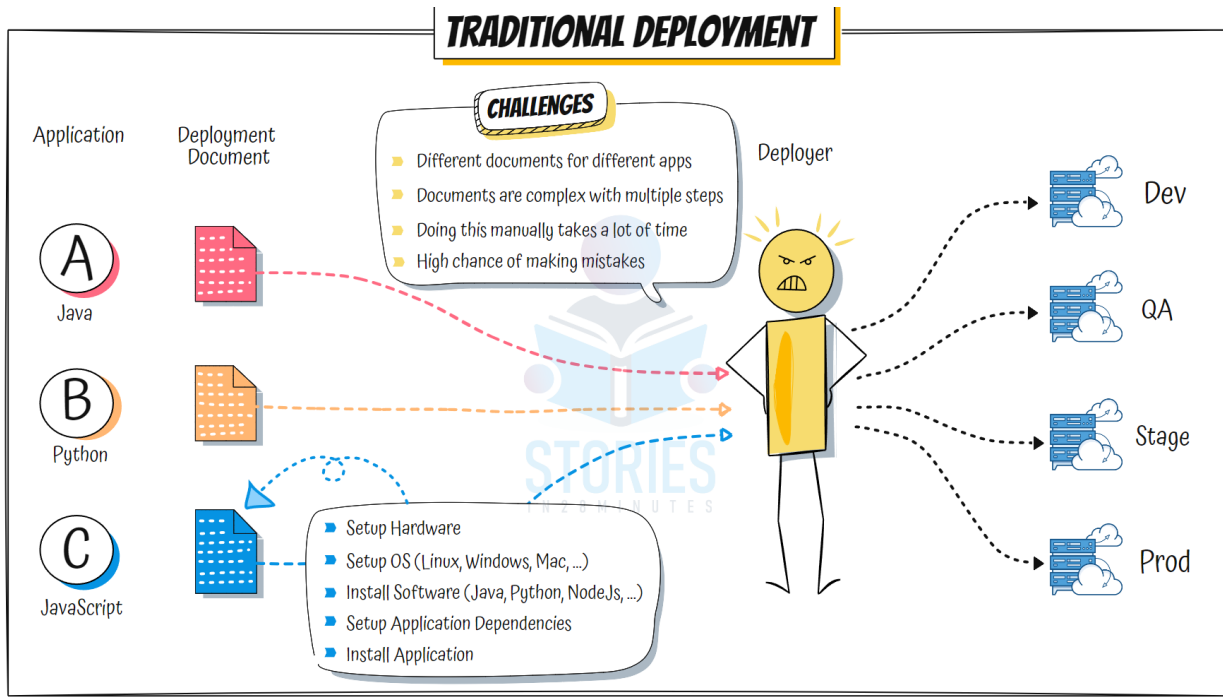
CHAPTER 10

Docker Interview - Diagrams

10 Questions & Answers

Q1

Traditional Deployment



Traditional deployment

Q2

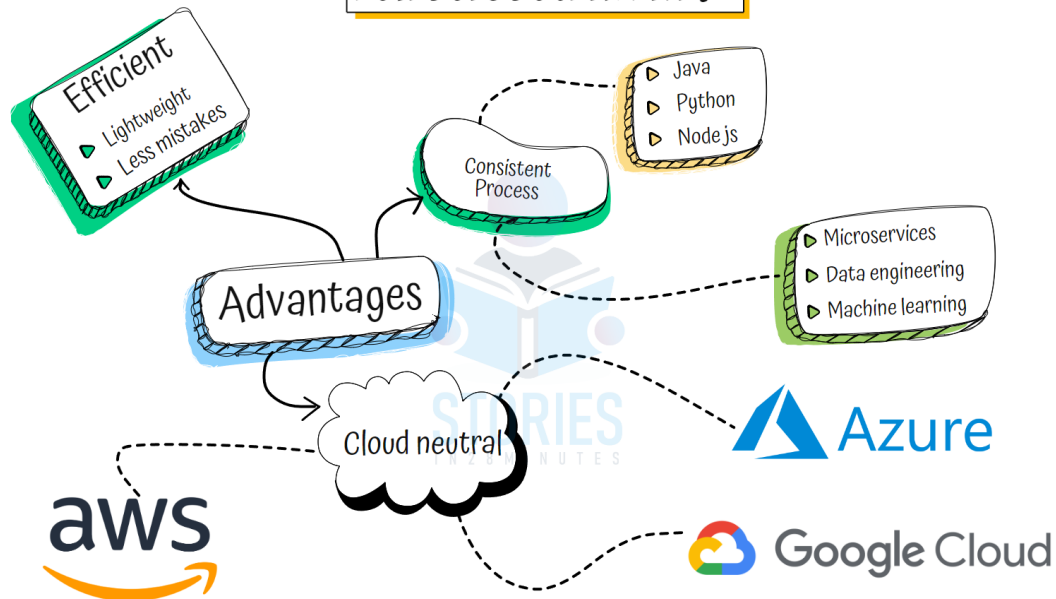
Advantages of Docker

Advantages of Docker

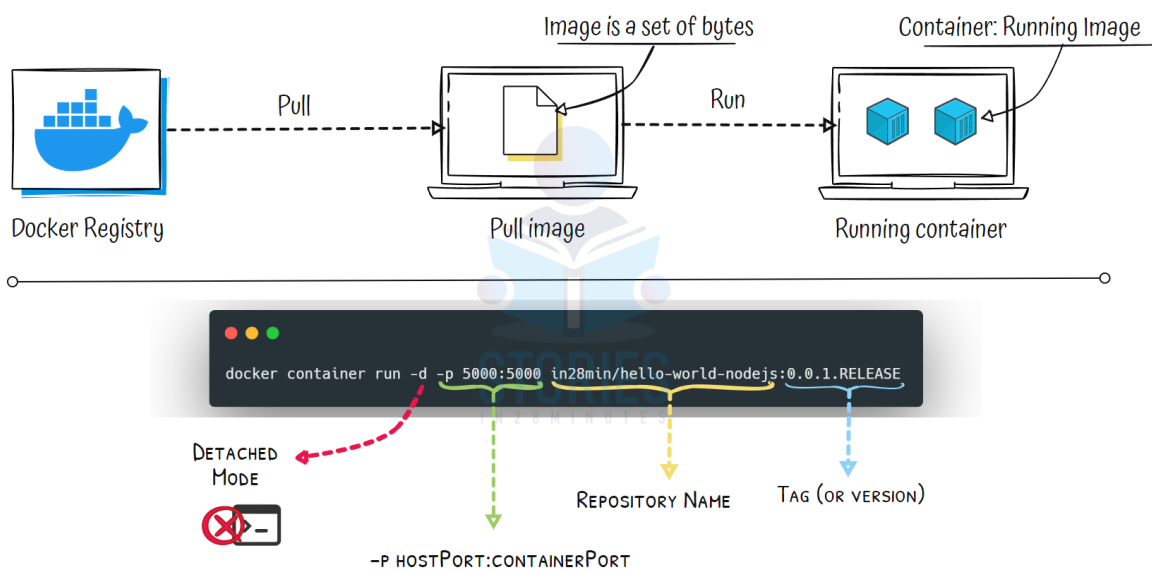
Q3

Running Docker Image

HOW DOES DOCKER HELP?



RUNNING A DOCKER IMAGE

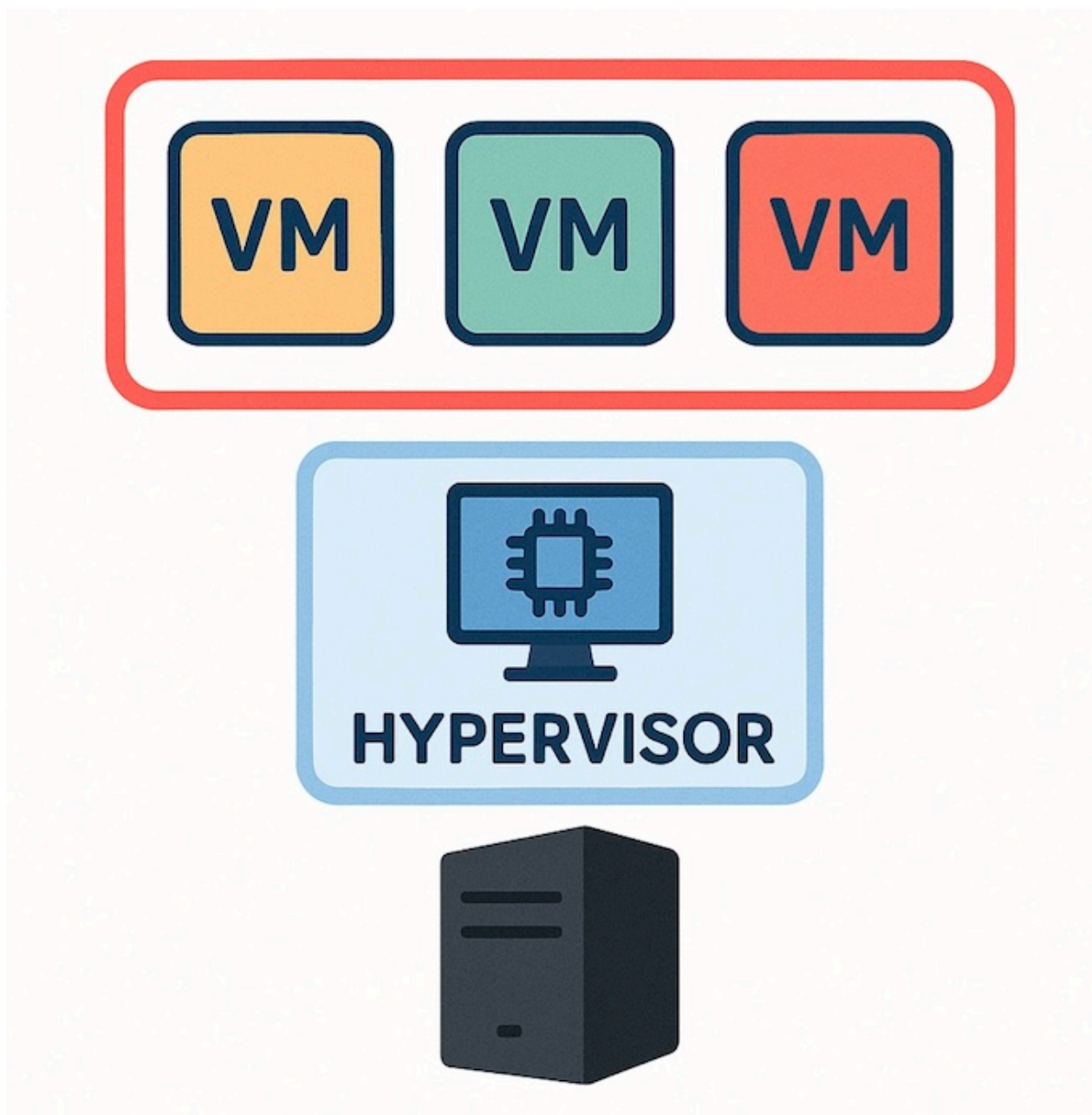


Running Docker Image

Q4

Virtual Machine Architecture

Virtual Machine Architecture



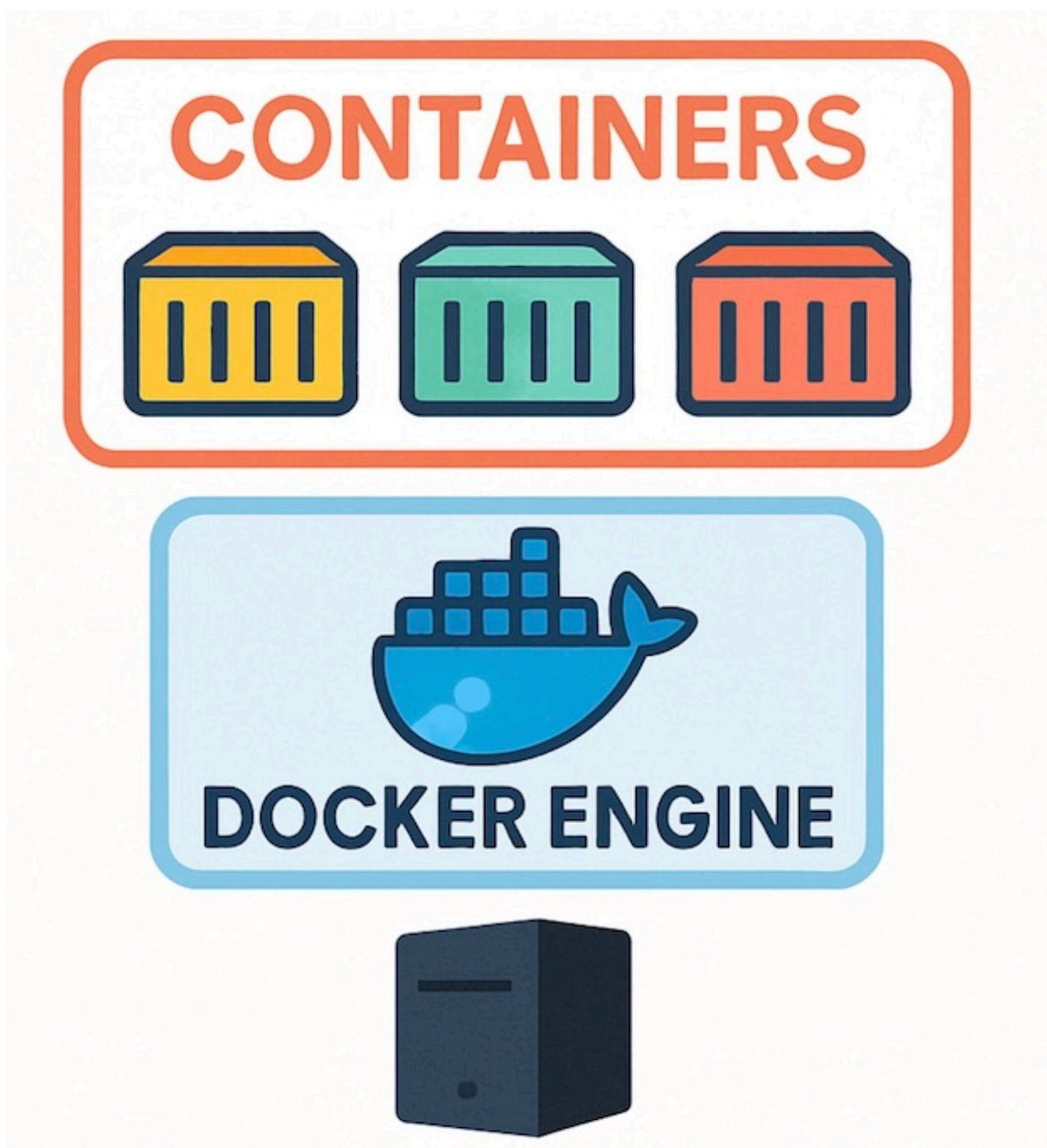
Q5

Container Architecture

Container Architecture

Q6

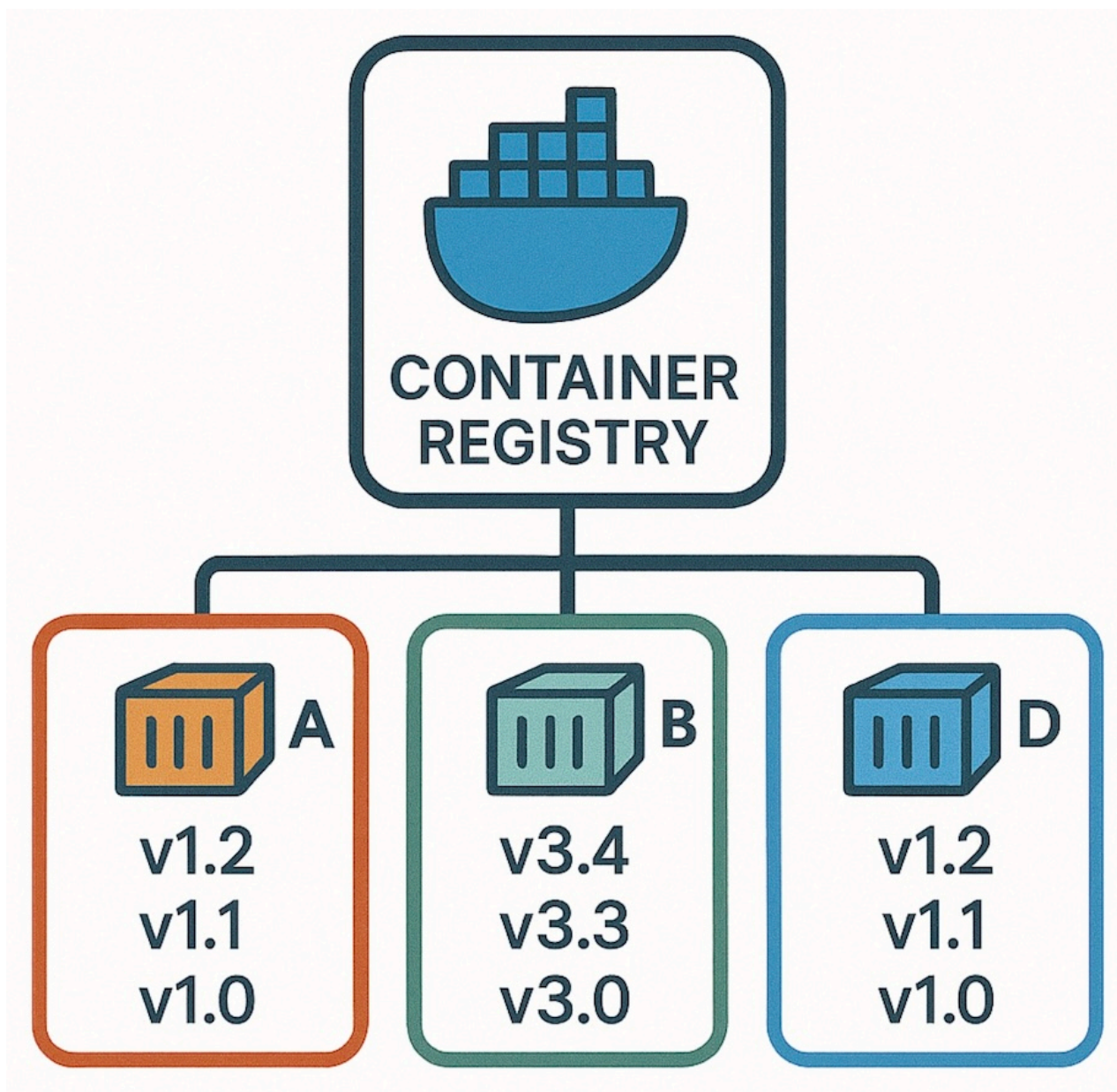
Container Registry



Container Architecture

Q7

Dockerfile



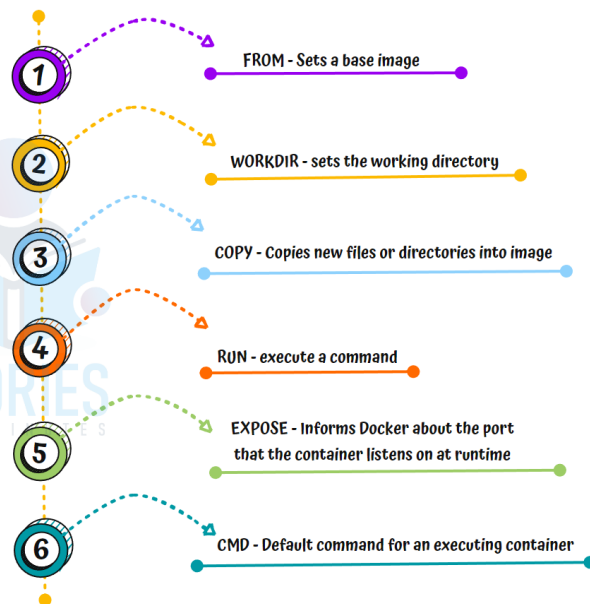
Dockerfile

Q8

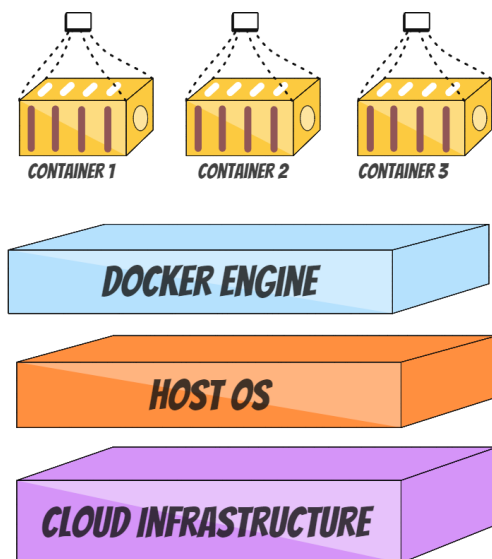
Why is Docker Popular

CREATING DOCKER IMAGES - DOCKERFILE

```
Dockerfile
FROM node:8.16.1-alpine
WORKDIR /app
COPY . /app
RUN npm install
EXPOSE 5000
CMD node index.js
```



WHY IS DOCKER POPULAR?



- Standardized Application Packaging:** Docker provides a uniform packaging mechanism for different types of applications, whether they are written in Java, Python, or JavaScript (JS).
- Multi-Platform Support:** Docker can be used across various platforms, including a local machine, data centers, and cloud platforms like AWS, Azure, and Google Cloud.
- Isolation:** Docker containers offer isolation from one another, ensuring that processes and applications run in separate environments without affecting each other.
- Standardization:** Simplified Operations. Consistent deployment, monitoring, logging ...
- Efficient:** Docker containers are lightweight. Compared to Virtual Machines as they do not have a Guest OS.

Why is Docker Popular

Q9

Docker Image Creation and Use

Dockerfile

```
sequenceDiagram
    participant Dev as Developer
    participant CLI as Docker CLI
    participant Daemon as Docker Daemon
    participant Hub as DockerHub

    %% Step 1: Docker Image Build
    Dev->>CLI: docker build -t my-app .
    CLI->>Daemon: Build image request
    Daemon->>Daemon: Build the image locally
    Daemon->>CLI: Build complete
    CLI->>Dev: Image created successfully

    %% Step 2: Push to DockerHub
    Dev->>CLI: docker push my-app
    CLI->>Daemon: Push image request
    Daemon->>Hub: Authenticate and push layers
    Hub-->>Daemon: Acknowledge image pushed
    Daemon->>CLI: Push complete
    CLI->>Dev: Image pushed successfully

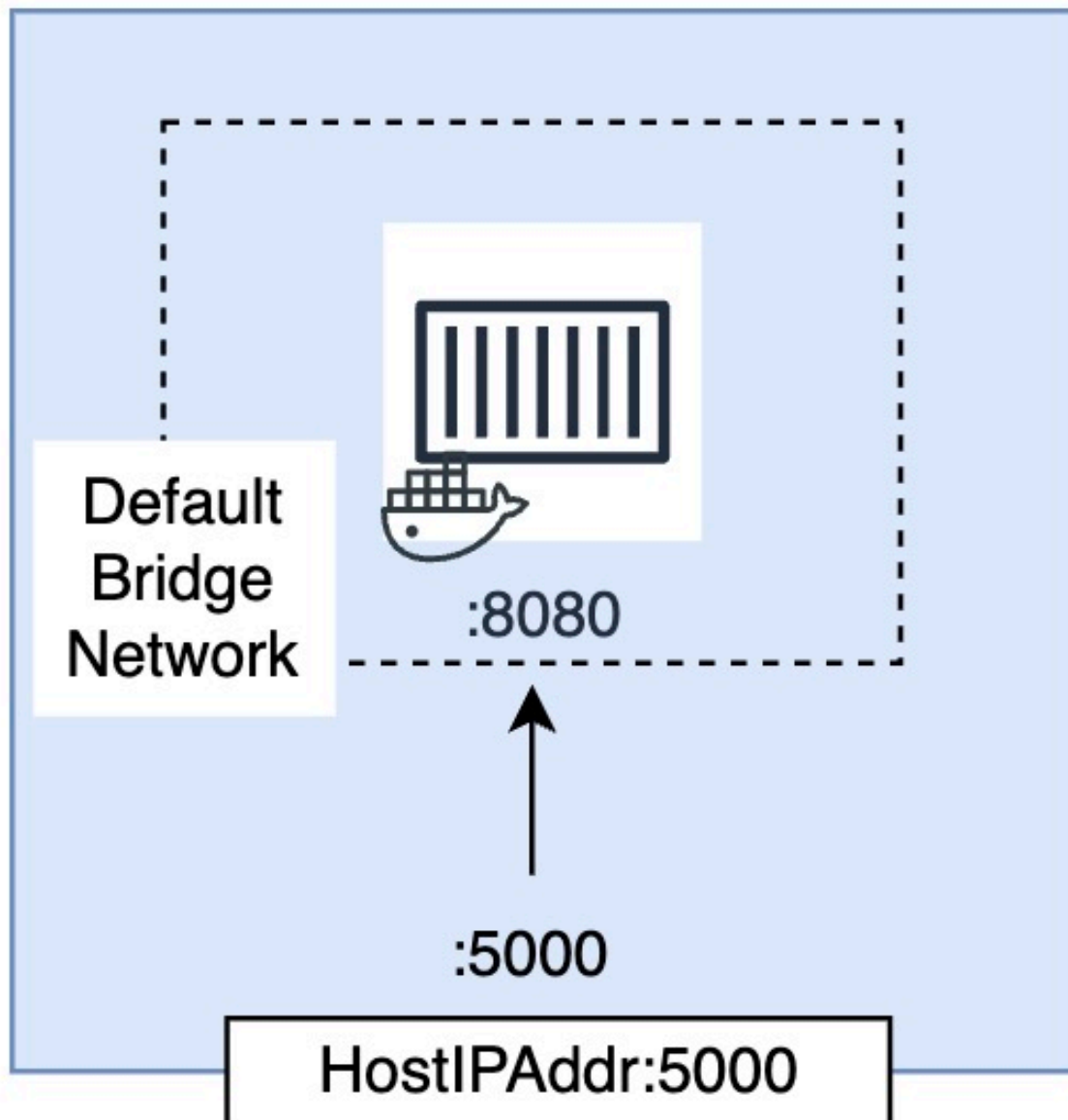
    %% Step 3: Pull from DockerHub
    Dev->>CLI: docker pull my-app
    CLI->>Daemon: Pull image request
    Daemon->>Hub: Authenticate and request image
    Hub-->>Daemon: Send image layers
    Daemon->>CLI: Pull complete
    CLI->>Dev: Image pulled successfully

    %% Step 4: Run Docker Container
    Dev->>CLI: docker run my-app
    CLI->>Daemon: Run container request
    Daemon->>Daemon: Allocate resources and start container
    Daemon->>CLI: Container started
    CLI->>Dev: Container is running
```

Q10

Docker Networking

Docker Bridge Network



Docker Host Network

